Politecnico di Torino

DDoS Attacks Detection and Characterization

Machine Learning for Networks — Project Report

Group Members

Bettini Alberto Bossio Francesca Lopiano Andrea Pesce Mattia

Politecnico di Torino

YEAR 2025-2026

Contents

1 Introduction

Internet security is a major challenge especially as the demand for online services keeps increasing. Distributed Denial of Service (DDoS) is one of the most disruptive threat because it is easy to deploy and very effective. This type of attack exploits many compromised machines, known as botnet, to generate large volumes of traffic that exhaust network or services resources, preventing legitimate users from accessing web-based applications.

In this project, we assume that different DDoS attacks generate different and recognizable traffic patterns. This motivates the use of Machine Learning (ML) techniques to automatically detect attacks and support network administrators with faster monitoring and countermeasures.

1.1 Dataset Overview

The provied dataset contains flow-level traffic traces collected during a controlled acquisition campaign that mixes benign user activity with multiple DDoS attacks. Each sample corresponds to a flow that is a sequence of packets exchanged between a source and a destination endpoint.

1.2 Attack classes

Below we briefly summarize each class:

- benign: legitimate background traffic generated by normal user activity (e.g., web, file transfer, email).
- ddos_dns: DNS reflection/amplification abusing open DNS resolvers;
- ddos_ntp: NTP reflection/amplification exploiting exposed NTP servers;
- ddos_snmp: SNMP reflection/amplification leveraging misconfigured SNMP devices/agents;
- ddos_ssdp: SSDP/UPnP reflection/amplification abusing SSDP discovery mechanisms;
- ddos_ldap: CLDAP (connectionless LDAP) reflection/amplification using UDP-based LDAP/CLDAP services;
- ddos_mssql: MSSQL reflection/amplification abusing exposed SQL Server resolution/discovery services;
- ddos_netbios: NetBIOS (NBNS) reflection/amplification abusing Internet-exposed NetBIOS name services;
- ddos_tftp: TFTP reflection/amplification exploiting exposed TFTP servers;
- ddos_udp: UDP flood sending large volumes of UDP packets to overwhelm bandwidth, CPU, or stateful firewall processing.
- ddos_udp_lag: UDP-lag traffic intended to disrupt perceived connectivity (commonly described in gaming scenarios) by inducing high latency/jitter and packet loss through UDP flooding patterns.
- ddos_syn: SYN flood exploiting the TCP three-way handshake by generating many half-open connections and exhausting server resources.

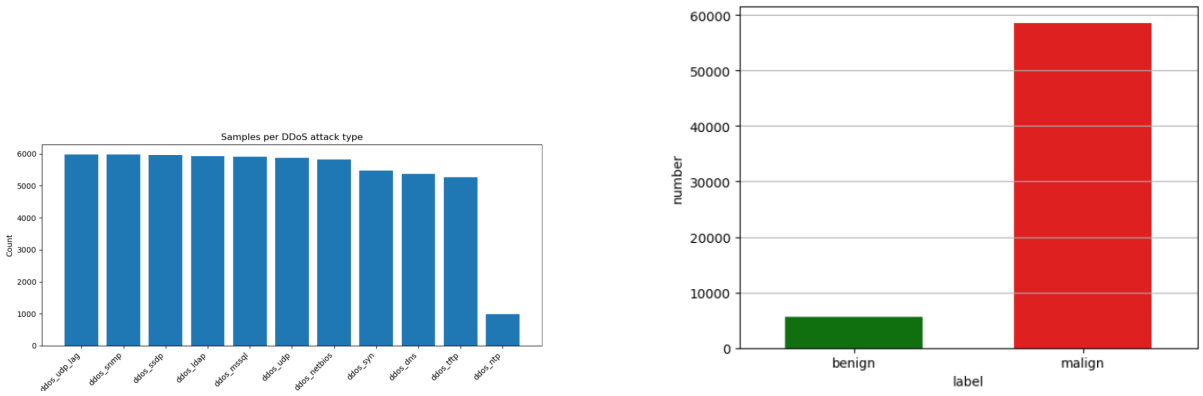
2 Data exploration and pre-processing

The dataset is composed of 64,239 flow samples and 88 columns (including identifiers and the ground-truth label). Each row represents a flow which is a sequence of packets exchanged between a source and a destination endpoint, summarized through flow-level statistics (packet counts, sizes, inter-arrival times, flags, activity/idle measures, etc.). The traffic capture spans approximately 4.29 hours, from 09:17 to 13:34, during which benign activity and multiple DDoS attacks were executed within the acquisition window.

Before starting we immediately notice that there were useless and redundant columns so we did as following After standardizing the column names and removing the spurious Unnamed: 0 index column, we further simplified the feature space by discarding non-informative variables. In particular, we identified all columns that assume the same value for every sample (i.e., features with a single unique value) and removed them, since they cannot contribute to discrimination between classes. Additionally, the feature SimilarHTTP was binarized by mapping any non-zero value to 1 and zero to 0. Overall, this preprocessing step reduces the dimensionality of the dataset resulting in still 64,239 rows but 74 columns.

2.1 Ground truth (GT) analysis

The ground truth contains 12 classes: one benign class and 11 DDoS classes. Figure **??** shows that the dataset is moderately imbalanced: most classes have a comparable number of flows, while ddos_ntp is significantly under-represented. This imbalance is important because it can affect both supervised training (bias toward frequent classes) and clustering evaluation (dominance of large groups). The dataset is strongly imbalanced toward attacks, with benign traffic representing only 5,658 flows (8.8%) versus 58,581 malicious flows (91.2%), meaning DDoS traffic is about 10× more frequent than normal traffic Figure **??**.



(a) Distribution of benign vs malicious traffic.

(b) Distribution of the 12 labels.

Figure 1: Label analysis of the DDoS dataset.

2.2 Generic traffic characterization

We first inspected the dataset at a generic traffic level to understand which protocol families and transport patterns dominate the flows. Figure **??** shows that the majority of flows are carried over UDP, with a smaller fraction over TCP. This is consistent with the presence of multiple volumetric and reflection-style attacks, which frequently rely on UDP due to its stateless nature.

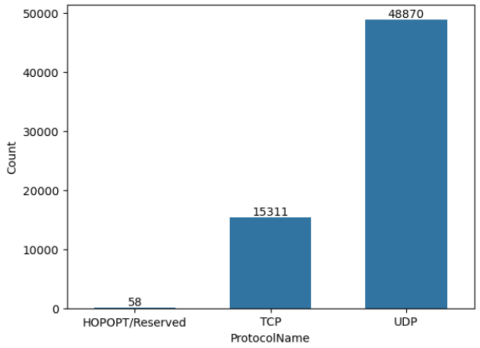


Figure 2: Protocol distribution (IP protocol numbers).

To analyze endpoint behavior, we studied the distribution of destination ports. Figure **??** reports the ECDF of destination ports for benign traffic and a subset of representative attacks. Benign flows concentrate more on well-known service ports, while attack traffic tends to be more spread toward high ports, suggesting heterogeneous targeting patterns.

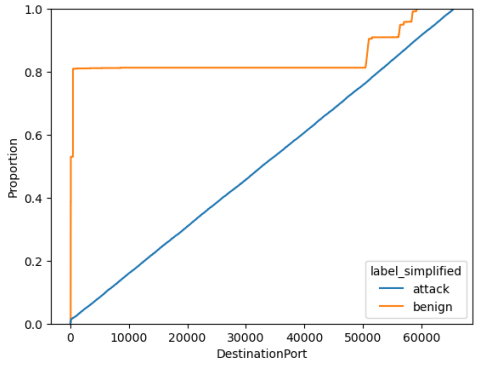
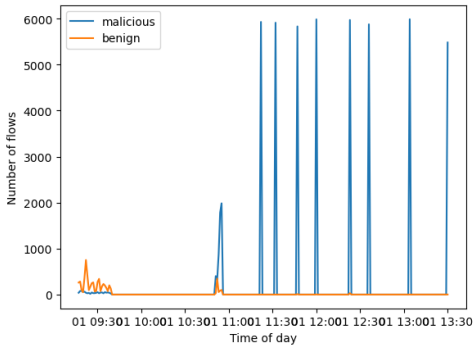


Figure 3: ECDF of destination ports for benign traffic and selected attacks.

We analyzed the temporal evolution of the traffic by splitting the capture window into 30-second slots and counting the number of flows in each interval. The resulting plot shows several spikes indicating short periods with a sudden increase in flow volume, which is consistent with DDoS activity where many flows are generated in bursts over limited time windows.



2.3 Source/Destination IP and Port characterization

We inspected the most frequent Source IP and Destination IP values to understand how traffic is distributed across endpoints. The overall counts are highly concentrated: a single source host

(172.16.0.5) generates 57,994 flows, while a single destination host (192.168.50.1) receives 58,000 flows. When restricting the analysis to malicious traffic, the concentration becomes even stronger, with the vast majority of attack flows exchanged between 172.16.0.5 and 192.168.50.1 . This suggests a controlled experimental setup with a dominant victim/target host and a dominant generator/reflection direction, hence raw IP addresses are highly dataset-specific and may cause severe overfitting if used as model features. In contrast, benign flows show a more diverse set of endpoints, including multiple internal clients and external public services like 4.2.2.4 and 8.8.8.8, which is consistent with normal user activity.

We also analyzed Source Port and Destination Port to capture which services are contacted and which transport patterns dominate. Benign traffic mainly targets standard application ports, especially 53 (DNS), 80 (HTTP), and 443 (HTTPS), while source ports appear more dispersed as expected for client-side ports. Attack traffic, instead, exhibits a more structured behavior: destination ports are less different and the distribution is dominated by a small set of values, whereas source ports concentrate on a few recurring ports (e.g., 900, 634, 648, 672), indicating repetitive generation patterns typical of automated DDoS traffic. Overall, port statistics provide useful signals for later tasks, while IP addresses mainly reflect the experimental topology and are therefore excluded from the final feature matrix used for learning.

2.4 Generate additional features

Given that the dataset already provides a rich set of flow-level aggregate statistics, we did not introduce additional engineered features and instead focused on selecting, cleaning, and scaling the existing ones.

2.5 Data pre-processing

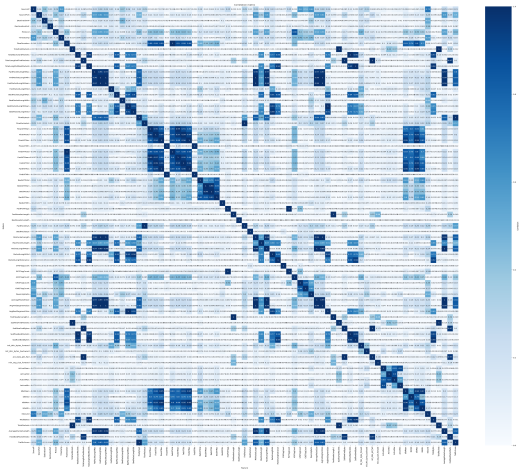
In order to build a consistent feature matrix for the subsequent supervised and unsupervised tasks, we applied a sequence of preprocessing steps involving time normalization, encoding of categorical fields, scaling, and redundancy removal.

Timestamp normalization. We converted the original timestamp into a relative time representation by selecting the minimum timestamp in the dataset as reference and replacing each timestamp with the elapsed time (in seconds) from that reference. This produces a comparable temporal feature that can be used for exploratory analysis and for time-based operations.

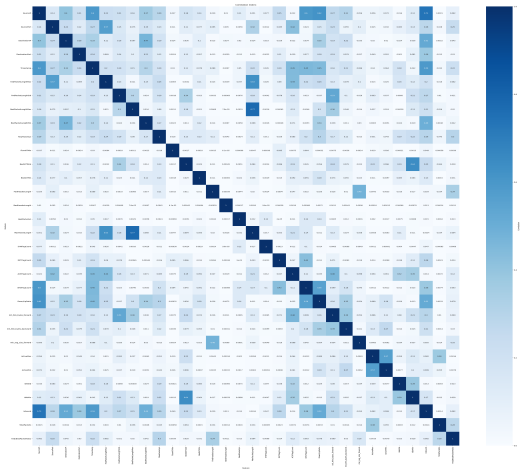
Categorical encoding. Since Machine Learning models require numerical inputs, we encoded the categorical fields using LabelEncoder. In particular, we transformed the ground-truth label into an integer target variable IntLabel. Additionally, we encoded SourceIP and DestinationIP into integers to allow their inclusion in the numerical preprocessing pipeline. We then kept the original string labels in a separate dataframe for interpretability and reporting.

Standardization. Because the retained features have heterogeneous units and ranges (counts, bytes, rates, and time-related statistics), we applied StandardScaler to obtain a standardized matrix. This step is essential for correlation analysis and PCA, and it also benefits distance-based methods used later in the project.

Correlation analysis and redundancy reduction. On the standardized matrix, we computed the absolute Pearson correlation between all pairs of features and visualized it using a heatmap **??**. To reduce redundancy, we selected highly correlated pairs using a threshold of $|r| > 0.8$. From these pairs, we constructed a set of features to remove by iteratively discarding one feature at a time and updating the candidate list, ensuring that each removal reduces the number of remaining strong correlations **??**. The identified features were dropped from the standardized matrix, producing the final preprocessed dataframe used for PCA and later tasks.

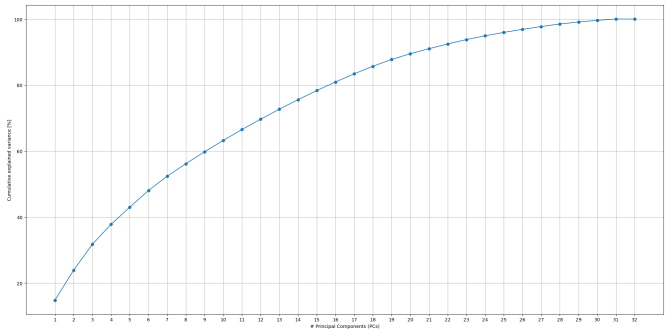


(a) Correlation Matrix Before.



(b) Correlation Matrix After.

PCA-based visualization. We applied Principal Component Analysis (PCA) on the decorrelated standardized matrix to assess the intrinsic dimensionality of the data and to support visualization. The plot below reports the cumulative explained variance as a function of the number of principal components (PCs). The curve increases rapidly for the first components and then gradually saturates: the first 10 PCs capture about two thirds of the total variance, around 15 PCs reach roughly 80%, and with 20 PCs the cumulative explained variance is 89.50% (i.e., nearly 90%). Beyond this point, the marginal gain per additional component becomes small, and more than 25 PCs are sufficient to exceed 95% of the variance.



3 Supervised learning – classification

In this section, we focus on the detection of DDoS attacks using supervised machine learning algorithms. The goal is to classify each sample in our dataset as benign or belonging to a specific attack class. We employ three distinct classifiers: Support Vector Machines (SVM), Random

Forest (RF) and k-Nearest Neighbors (kNN). We evaluate their performance under different data splitting strategies and optimize their hyper-parameters to achieve the best detection rates.

3.1 Splitting Strategies

To rigorously evaluate the generalization capability of our models, we implemented two different strategies for splitting the dataset into training and testing sets:

1. **Label Split**: The data is shuffled and split randomly, ensuring that the distribution of classes remains consistent in both the training and testing sets.

```
X_train_l, X_test_l, y_train_l, y_test_l = train_test_split(
    new_df, labels, stratify=labels, train_size=0.7, random_state=42)
```

2. **Temporal Split**: For each label, the samples are first sorted chronologically by the `Timestamp` feature. A train–test split is then applied within each category based on this ordering. The first portion of the data (70%) is used for training (`X_train_T` and `y_train_T`), and the subsequent portion is used for testing (`X_test_T` and `y_test_T`). This method simulates a real-world scenario where a model trained on past traffic must detect future attacks.

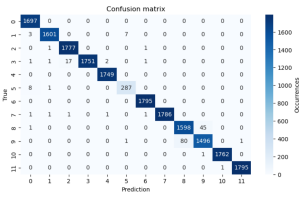
3.2 Training and Evaluation for the two different splits

For each trained model, we used the default parameters and performances are evaluated by means of confusion matrix and classification report. Since there are three models with two different splits we decided to insert just the weighted F_1-score for the test set, but all the metrics and plots are available on the Jupyter notebook.

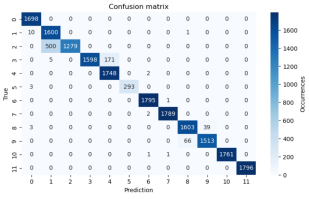
3.2.1 Support Vector Machine

This is a powerful classifier that finds the optimal hyperplane to separate classes. It is effective in high-dimensional spaces but can be computationally expensive. We started creating our *SVC()*, then we fitted it on the training sets and finally predicted on the training and test sets. On the training set we obtain 0.99 of accuracy with the label split and with the temporal split. These are the results on the test sets (the first two images are referred to label split and the last two to temporal split):

	precision	recall	f1-score	support
0	0.99	1.00	1.00	1697
1	1.00	0.99	1.00	1611
2	0.99	1.00	0.99	1779
3	1.00	0.99	0.99	1773
4	1.00	1.00	1.00	1749
5	0.97	0.97	0.97	296
6	1.00	1.00	1.00	1795
7	1.00	1.00	1.00	1791
8	0.95	0.97	0.96	1644
9	0.97	0.95	0.96	1578
10	1.00	1.00	1.00	1763
11	1.00	1.00	1.00	1796



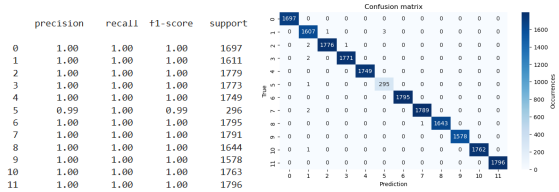
	precision	recall	f1-score	support
0	0.99	1.00	1.00	1698
1	0.76	0.99	0.86	1611
2	1.00	0.72	0.84	1779
3	1.00	0.90	0.95	1774
4	0.91	1.00	0.95	1750
5	1.00	0.99	0.99	296
6	1.00	1.00	1.00	1796
7	1.00	1.00	1.00	1791
8	0.96	0.97	0.97	1645
9	0.97	0.96	0.97	1579
10	1.00	1.00	1.00	1763
11	1.00	1.00	1.00	1796



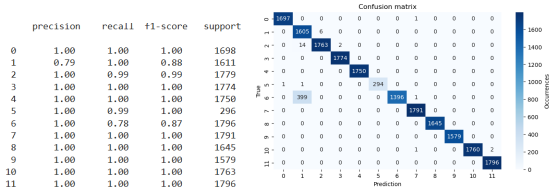
3.2.2 Random Forest

It is an ensemble learning method that constructs multiple decision trees during training. It is robust against overfitting and handles non-linear relationships well. We started creating our *RandomForestClassifier()*, then we fitted it on the training sets and finally predicted on the training and test sets. On the training set we obtain 1.00 of accuracy with the label split and

with the temporal split. These are the results on the test sets (the first two images are referred to label split and the last two to temporal split):



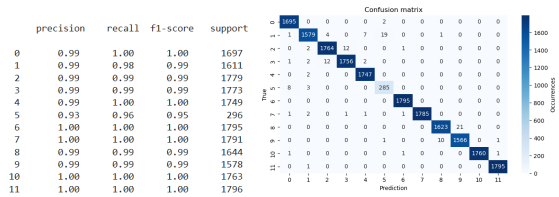
	precision	recall	f1-score	support
0	1.00	1.00	1.00	1697
1	1.00	1.00	1.00	1611
2	1.00	1.00	1.00	1779
3	1.00	1.00	1.00	1773
4	1.00	1.00	1.00	1749
5	0.99	1.00	0.99	296
6	1.00	1.00	1.00	1795
7	1.00	1.00	1.00	1791
8	1.00	1.00	1.00	1644
9	1.00	1.00	1.00	1578
10	1.00	1.00	1.00	1763
11	1.00	1.00	1.00	1796



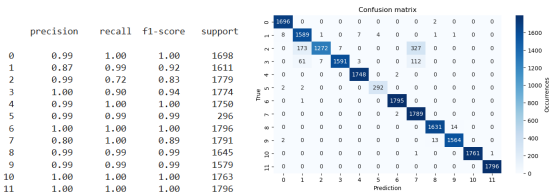
	precision	recall	f1-score	support
0	1.00	1.00	1.00	1698
1	0.79	1.00	0.88	1611
2	1.00	0.99	0.99	1779
3	1.00	1.00	1.00	1774
4	1.00	1.00	1.00	1750
5	1.00	0.99	1.00	296
6	1.00	0.78	0.87	1796
7	1.00	1.00	1.00	1791
8	1.00	1.00	1.00	1645
9	1.00	1.00	1.00	1579
10	1.00	1.00	1.00	1763
11	1.00	1.00	1.00	1796

3.2.3 k-Nearest Neighbors

It is a method that classifies a sample based on the majority class of its k nearest neighbors. It is simple but sensitive to feature scaling. We started creating our *KNeighborsClassifier()*, then we fitted it on the training sets and finally predicted on the training and test sets. On the training set we obtain 1.00 of accuracy with the label split and with the temporal split. These are the results on the test sets (the first two images are referred to label split and the last two to temporal split):



	precision	recall	f1-score	support
0	0.99	1.00	1.00	1697
1	0.99	0.98	0.99	1611
2	0.99	0.99	0.99	1779
3	0.99	0.99	0.99	1773
4	0.99	1.00	1.00	1749
5	0.93	0.96	0.95	296
6	1.00	1.00	1.00	1795
7	1.00	1.00	1.00	1791
8	0.99	0.99	0.99	1644
9	0.99	0.99	0.99	1578
10	1.00	1.00	1.00	1763
11	1.00	1.00	1.00	1796



	precision	recall	f1-score	support
0	0.99	1.00	1.00	1698
1	0.87	0.99	0.92	1611
2	0.99	0.72	0.83	1779
3	1.00	0.90	0.94	1774
4	0.99	1.00	1.00	1750
5	0.99	0.99	0.99	296
6	1.00	1.00	1.00	1796
7	0.80	1.00	0.89	1791
8	0.99	0.99	0.99	1645
9	0.99	0.99	0.99	1579
10	1.00	1.00	1.00	1763
11	1.00	1.00	1.00	1796

We can see that the performance vary among different attacks. This is mainly because certain attacks (like ddos_snmp and ddos_dns) have features that either overlap significantly with other traffic or evolve over time, making them harder to generalize in a **temporal split** scenario. In contrast, the **label split** effectively "leaks" information by mixing packets from the same time bursts into both training and testing, artificially smoothing out these variations.

By analyzing the confusion matrices and classification reports, it is possible to understand if underfitting or overfitting occur. Underfitting typically presents as poor accuracy, recall, and F_1-score on both training and test sets, accompanied by high off-diagonal elements in the confusion matrix for the training set. In contrast, overfitting is observed when the model achieves high accuracy, precision, recall, and F_1-score on the training set but lower values on the test set and diagonal elements during training but fails to replicate them on the test set. Based on these indicators, it can be concluded that neither phenomenon is present.

Underfitting is characterized by low accuracy, recall, and F_1-score on both training and test sets, On the other hand, overfitting manifests as , accompanied by high diagonal elements in the confusion matrix for the training set. Based on those statements, it is possible to declare that neither underfitting nor overfitting occur.

To know which model generates the best performance, we decided to compute the weighted F_1-score on all test sets and these are the results:

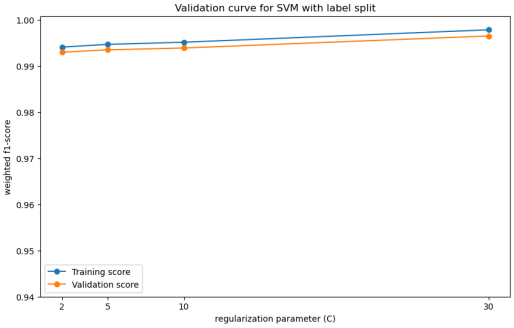
```
Weighted F1-score (label split) - SVM: 0.99076    RF: 0.99933    kNN: 0.99368
Weighted F1-score (temporal split) - SVM: 0.95787    RF: 0.97853    kNN: 0.96012
```

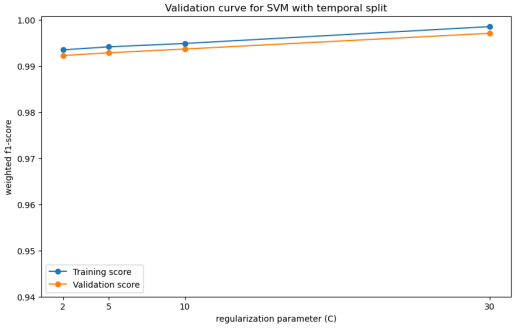
So, we can see that the model with the best performance is the Random Forest with the label split. In fact we have seen that this kind of split leaks information and so obtains a better performance.

3.3 Hyper-parameters tuning

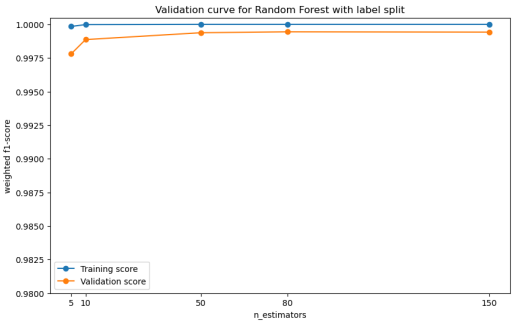
We utilized validation curves to tune key hyper-parameters. The tuning process aimed to balance training cost and performance.

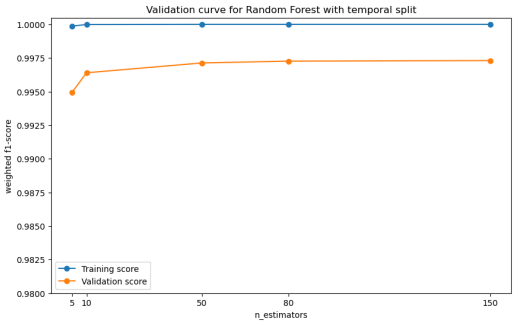
In particular, for the SVM we investigated the regularization parameter C. Higher values of C aim to classify all training examples correctly (hard margin), while lower values allow for some misclassifications (soft margin). The optimal value was found to be $C = 5$.



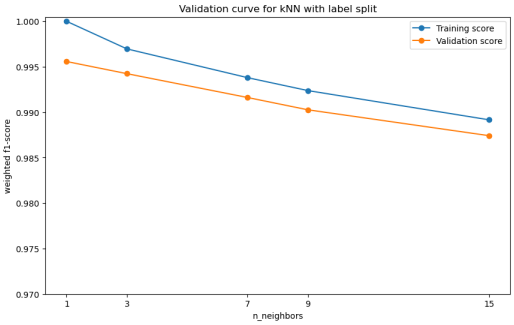


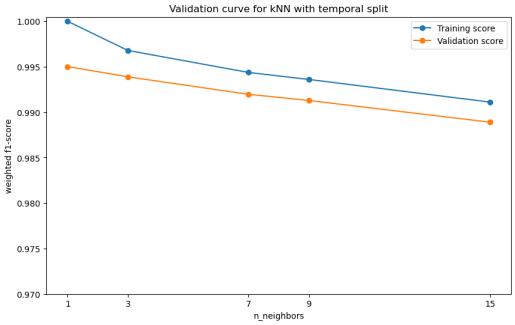
For the Random Forest we tuned the number of trees in the forest (`n_estimators`). While increasing the number of trees generally improves performance, it also increases computational cost. We found that `n_estimators=10` provided a strong balance between speed and accuracy, although models with more estimators (for example the default value for `n_estimators` we used before tuning is 100) yielded marginally better results.





For the kNN we varied the number of neighbors k (`n_neighbors`). Small values of k can lead to overfitting (noise sensitivity), while large values can smooth out the decision boundary too much. The optimal performance was achieved with `n_neighbors=3`.





Now we create and train the same models but with the hyper-parameters we identified above. These are the results on the test set compared to those obtained before tuning:

```
Before tuning:
Weighted F1-score (label split) - SVM: 0.99076    RF: 0.99933    kNN: 0.99368
Weighted F1-score (temporal split) - SVM: 0.95787    RF: 0.97853    kNN: 0.96012
After tuning:
Weighted F1-score (label split) - SVM: 0.99294    RF: 0.998497    kNN: 0.99461
Weighted F1-score (temporal split) - SVM: 0.95884    RF: 0.96356    kNN: 0.96114
```

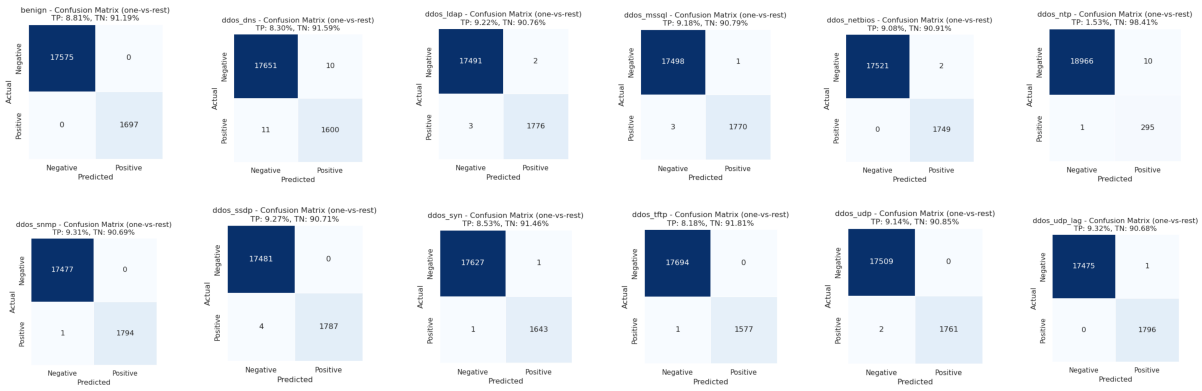
All the results after tuning of hyper-parameters are better than the previous ones. Only the Random Forest has slightly higher values with n_estimators=100 than with n_estimators=10, as we might expect.

The best model in terms of performance is the Random Forest with n_estimators=100. While the fastest model is the Random Forest with n_estimators=10.

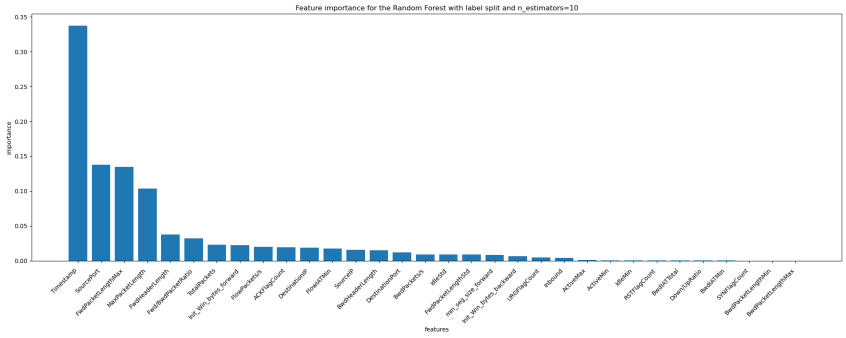
In general, the best model according to training cost and performance is the Random Forest with n_estimators=10.

3.4 False Positive and False Negative investigation

An investigation into misclassifications was conducted by evaluating False Positive and False Negative assignments for each class. The results indicate that errors are exceptionally rare across all labels, demonstrating the model's high accuracy in maximizing true positives while minimizing false assignments. This robust behavior reinforces the overall effectiveness of the classifier. Specifically, the scarcity of False Positives and False Negatives highlights the model's superior precision and recall capabilities.



Finally, the feature importance analysis indicates that the feature contributing the most to misclassification is the Timestamp.



Talking about a real DDoS detection context, we would have two main categories: benign (normal traffic) and malicious (DDoS labels like ddos_dns, ddos_udp, etc.). False Positive means "false alarm", so the model incorrectly predicts legitimate, normal traffic as an attack (DDoS). False Negative means "missed attack", so the model incorrectly predicts malicious traffic as normal (benign).

If we look at the precision, computed as $p = \frac{TP}{TP+FP}$, and we see that it is low, this means we have high False Positives (we are generating many false alarms). On the other hand, if the recall $r = \frac{TP}{TP+FN}$ is low, it means we have high False Negatives (we are missing attacks).

4 Unsupervised learning - Clustering

In this section, the main goal is to apply unsupervised learning techniques to cluster the dataset based on its intrinsic characteristics. We use the previously created new_df dataset for this analysis.
First, we perform a 2D PCA to visualize the data in a bidimensional space ($PC1, PC2$), defining a `plot_clustering` function for consistent visualization. Given the dataset's size, we initially extracted a representative sample of 10,000 records using the resample function to optimize computation times during the parameter exploration phase. Specifically, we extracted both features (`X_sample`) and their corresponding ground truth labels (`y_sample`), allowing us to perform both internal and external validation. However, for the final cluster analysis, the entire `new_df` dataset was used to ensure maximum precision.

4.1 KMeans

The first algorithm applied was K-means, an "hard" clustering technique wherein each sample is exclusively associated with a single cluster. This method partitions a set of observations into k clusters, where each observation is assigned to the cluster whose mean is closest to it.

4.1.1 Determing the number of cluster by Elbow method and Silhouette analysis

To identify the optimal number of clusters (k), we evaluated the algorithm's performance across a range from $k = 2$ to $k = 12$. The selection was guided by three complementary approaches:

- **Inertia(Elbow Method)**:Measuring the sum of squared distances to the closest center to find the "elbow" point.

- **Sillhouette Score(Euclidian)**:Evaluating cluster cohesion using standard straight-line distance.It ranges from -1 to 1, where higher values indicate better-defined clusters.
- **ARI (Adjusted Rand Index) & RI (Rand Index)**:External metrics used to measure the agreement between our unsupervised clusters and the actual attack labels (`y_sample`). ARI is especially valuable as it corrects the score for chance.

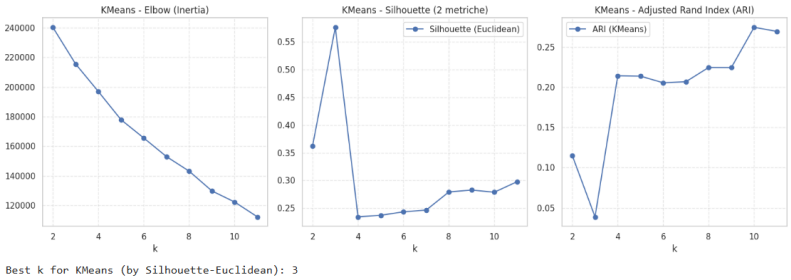
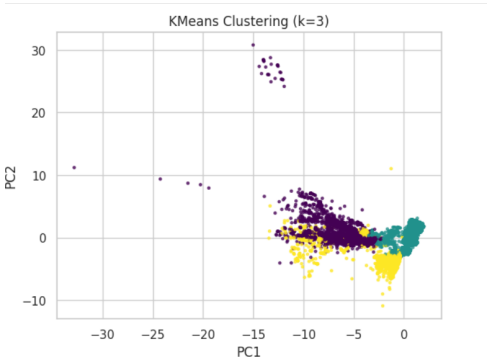


Figure 12: Multi-metric evaluation for KMeans cluster selection (k).

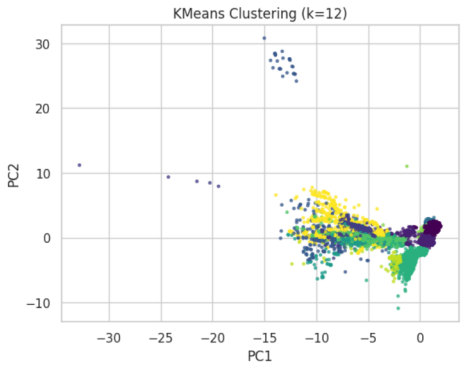
After determining the optimal number of clusters from the sampled data, we fitted the KMeans algorithm to the entire `new_df` dataset. To gain a deeper understanding of the results, we performed a comparative visualization between two different configurations:

- Optimal Clustering ($k = best\_k$): We applied the algorithm using the k value suggested by the maximum Silhouette score and ARI analysis. This represents the most statistically significant partitioning based on the data's intrinsic features.
- Label-Based Clustering ($k = 12$): We also executed KMeans with $k = 12$ to align the model with the actual number of ground truth categories (various attack types and benign traffic).

Using the `plot_clustering` function, we projected these results onto the 2D PCA space. This comparison allows us to evaluate the granularity of our clusters and visually assess if the unsupervised model can naturally distinguish between the known attack classes.



(a) Kmeans with k=3



(b) Kmeans with k=12

Figure 13: Visual comparison between optimal clustering and label-based clustering for KMeans

4.2 Gaussian Mixture Modeling (GMM)

To refine the analysis , we implemented GMM, a probabilistic approach to clustering. Unlike K-Means, GMM is a "soft" clustering technique, it assumes that the data is generated from a

mixture of several Gaussian distributions with unknown parameters, allowing a more flexible cluster shape.

4.2.1 Components Selection

Following the same logic applied to K-Means, we evaluated the Gaussian Mixture Model (GMM) performance for k values ranging from 2 to 12. The choice of the optimal number of components was guided by Silhouette Score (Euclidean), Adjusted Rand Index (ARI) and Random Index(RI). After selecting the best k via Silhouette score, we compared it with $k = 12$. The 2D PCA plots highlight GMM's ability to manage overlapping traffic flows and high-density regions more effectively than the distance-based KMeans.

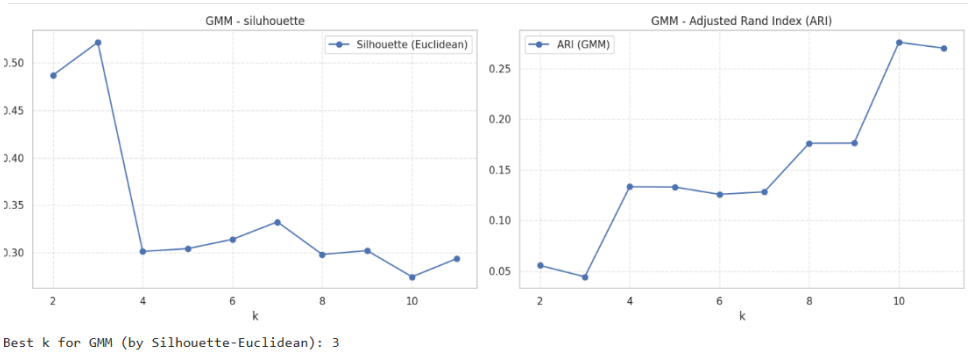
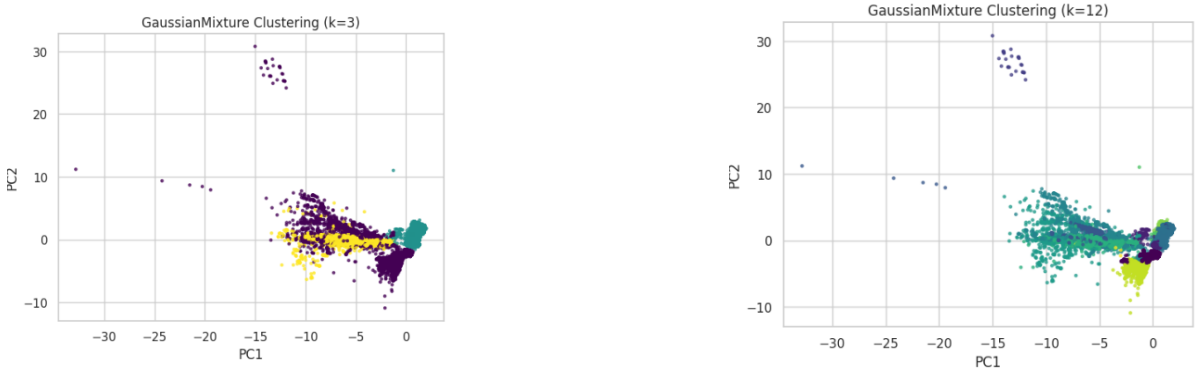


Figure 14: Multi-metric evaluation for GMM cluster selection (k).



(a) Kmeans with k=3

(b) Kmeans with k=12

Figure 15: Visual comparison between optimal clustering and label-based clustering for GMM

4.3 Performance Tuning

To ensure the robustness and quality of our clusters we performed a hyperparameter tuning phase for both KMeans and GMM. While the previous steps helped us identify the optimal number of clusters, this phase focuses on refining the internal parameters of the algorithms.

4.3.1 KMeans Tuning

For the Kmeans we explored different combinations of initialization methods (`k-means++` vs `random`) and number of initializations (`n_init` values of 10, 20, and 50). These configurations

were evaluated using the Euclidean Silhouette Score on our representative sample. Then the best setup was selected to perform the best final fit on the entire `new_df` dataset, ensuring that our final labels (`labels_km_full`) are the result of the most stable and well-separated model.

4.3.2 GMM Fine-Tuning

A similar approach was applied to the Gaussian Mixture Model, focusing on the covariance type and the number of components. The `covariance_type` is a critical parameter that defines the degrees of freedom for the shape and orientation of each cluster: **Full** (flexible ellipses), **Tied** (shared shapes), **Diag** (axis-aligned ellipses), or **Spherical** (circles).

To ensure a stable starting point we initialized the GMM using KMeans results(`init_params="kmeans"`). Then the final configuration was selected by maximizing the Silhouette score on the sample and then used to generate the definitive labels (`labels_gmm_full`) on the entire dataset.

	model	k	init	n_init	inertia	sil_euc
0	KMeans	2	k-means++	10	227159.679992	0.576335
1	KMeans	2	k-means++	20	227159.679992	0.576335
2	KMeans	2	k-means++	50	227159.679992	0.576335
3	KMeans	2	random	10	227159.679992	0.576335
4	KMeans	2	random	20	227159.679992	0.576335
5	KMeans	2	random	50	227159.679992	0.576335
6	KMeans	3	k-means++	10	207997.074676	0.571470
7	KMeans	3	k-means++	20	207997.074676	0.571470
8	KMeans	3	k-means++	50	207997.074676	0.571470
18	KMeans	5	k-means++	10	177248.304761	0.333226

(a) Tuning score table for Kmeans

	model	k	covariance_type	sil_euc
5	GMM	3	tied	0.571820
4	GMM	3	full	0.521676
6	GMM	3	diag	0.498337
0	GMM	2	full	0.486976
7	GMM	3	spherical	0.415360
3	GMM	2	spherical	0.415336
1	GMM	2	tied	0.357690
2	GMM	2	diag	0.356665
14	GMM	5	diag	0.304342
12	GMM	5	full	0.304195

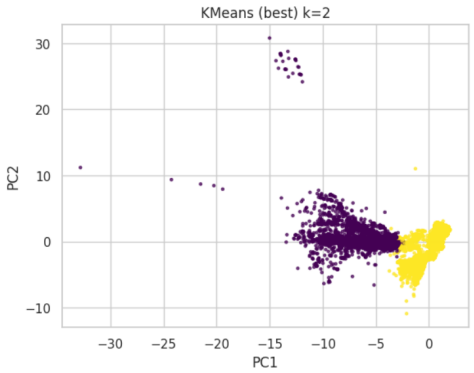
(b) Tuning score for GMM

Figure 16: Tuning score tables for Kmeans and GMM

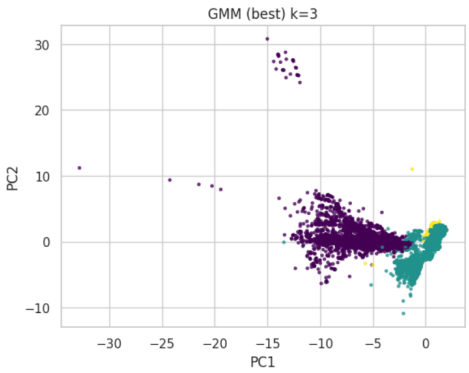
The optimization concluded that the best configuration for KMeans was $k = 2$, with "`kmeans++`" initialization and `n_init` $= 10$, achieving a Silhouette Score of 0.576335. However, the tuning process for GMM identified the "`tied`" covariance type as the most effective, with $k = 3$ components, resulting in an optimal probabilistic fit for the network traffic structure.

4.3.3 Clustering Evaluation

To determine which model best captured the underlying structure of the DDoS traffic, we performed a final comparative evaluation on the full dataset (`Xfull`) using the `eval_clustering` function. This utility calculates both internal and external metrics: Silhouette score, ARI(Adjusted Rand Index) and RI (Rand Index).



(a) Optimized Kmeans



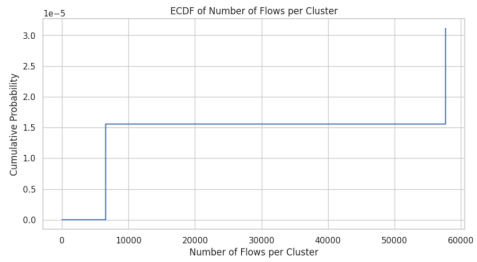
(b) Optimized GMM

Figure 17: Final performance comparison between optimized KMeans and GMM models

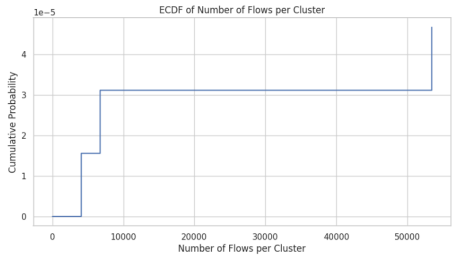
4.4 Coarse Analysis of the Detected Clusters

To conclude the analysis, we carried out a detailed inspection of the structure of the clusters identified ECDF and Silhouette analysis.

- **ECDF Analysis**: We calculated the distribution of cluster sizes (number of flows per cluster). These plots allow us to verify if the traffic is distributed into balanced groups or if there are "dominant" clusters capturing the majority of the data, which often corresponds to massive attack patterns.
- **Silhouette Plot**: This visualization shows the consistency of each individual point within its cluster. While the ECDF focuses on size, the Silhouette plot reveals the quality of the grouping: wider and more uniform profiles indicate cohesive and well-separated clusters, whereas narrow or negative values highlight overlaps between different traffic types.

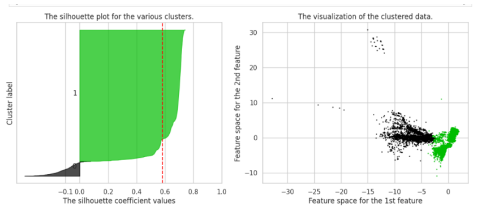


(a) ECDF of KMeans

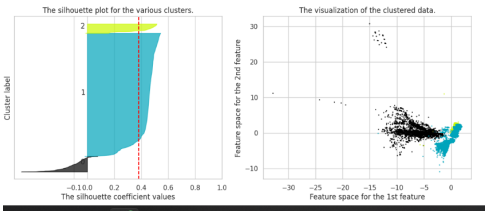


(b) ECDF of GMM

Figure 18: ECDF of flow distribution across clusters for both models



(a) Silhouette of KMeans



(b) Silhouette of GMM

Figure 19: Detailed Silhouette profiles and cluster density visualization for the final models

5 Clusters explainability and analysis

In this section we provide an explainability-driven analysis of the clustering results obtained in Section 3. We study how GT traffic types distribute across clusters, identify the most influential features for cluster separation, and investigate attack-specific patterns and similarities.

5.1 Cluster Distribution of Traffic Types

In this section we analyze how GT traffic labels are distributed across the clusters from Section 3. The goal is to assess cluster interpretability. We check cluster purity and class fragmentation. We also verify if benign and attack flows overlap.

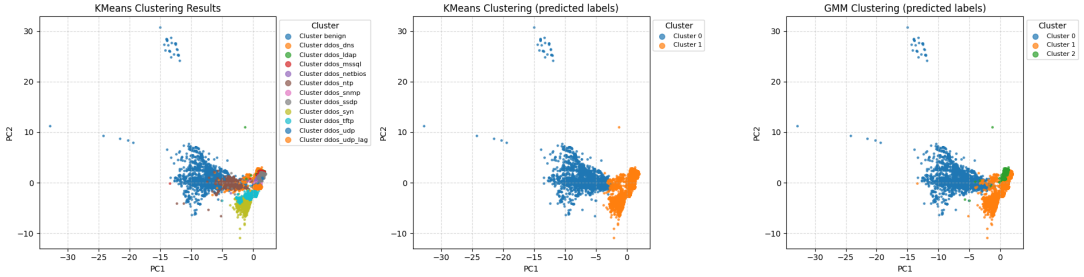


Figure 20: PCA scatter with predicted clusters.

We compute a contingency table between predicted cluster id and GT labels. We report a row-normalized version (cluster composition). Each row sums to 1. This shows class proportions inside each cluster. High dominance of one class implies higher purity. We also inspect how each GT class is split across clusters.

5.1.1 KMeans (k = 2)

KMeans creates two macro-clusters. One cluster is mainly benign, while the other aggregates almost all DDoS classes. This indicates a coarse separation, close to "benign vs attack". Attack-type characterization is limited, because many attack families share the same cluster. A minor overlap appears for `ddos_ntp`, with a small fraction assigned to the benign-dominated cluster.

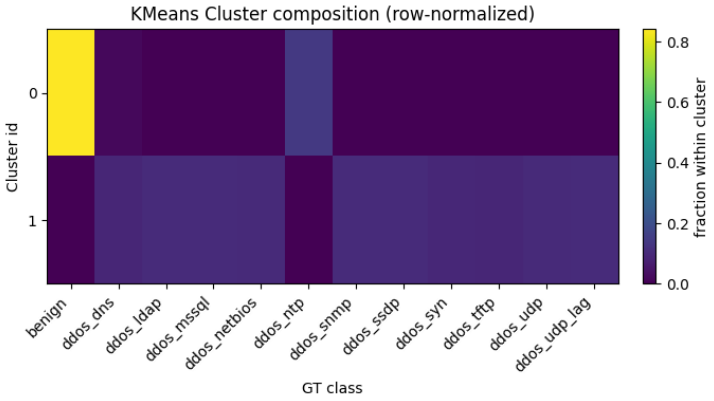


Figure 21: KMeans cluster composition (row-normalized).

5.1.2 GMM (3 components)

GMM provides a more structured partition. One component is benign-dominated. One component contains most attack types together. A third component is strongly associated with `ddos_dns`, suggesting that DNS attacks have a more distinct distribution. Some DNS samples still fall in the general attack cluster, indicating intra-class variability. `ddos_ntp` again shows partial overlap with benign.

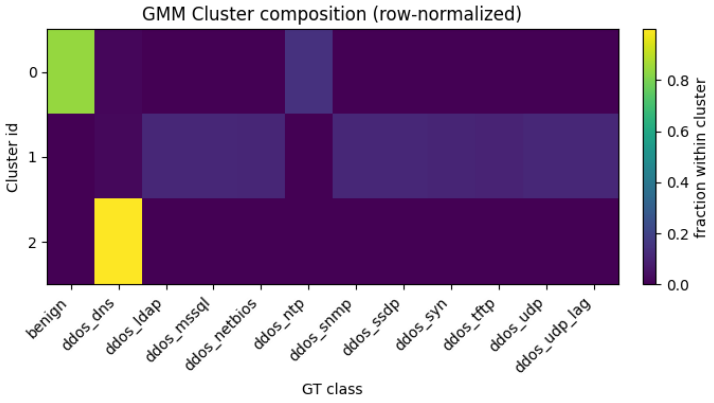


Figure 22: GMM cluster composition (row-normalized).

5.2 Feature Importance Analysis

In this section we identify which features are the most relevant to separate the clusters obtained in Section 3. Since clustering is unsupervised, we do not use GT labels. We instead explain the clustering outcome by measuring how input features contribute to predict the cluster assignment.

5.2.1 Method (Permutation Importance on a surrogate model)

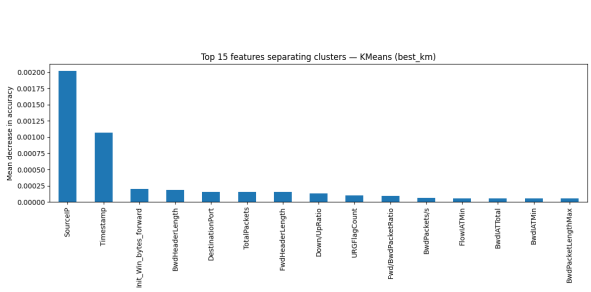
We train a supervised surrogate classifier to predict the cluster id from the original feature space. We then apply permutation importance on a held-out test set. For each feature, we randomly permute its values and measure the decrease in prediction accuracy. A larger accuracy drop indicates a more important feature for cluster separation. We report the Top-15 features. We also show the distribution of importance scores across repeated permutations.

5.2.2 KMeans (k = 2)

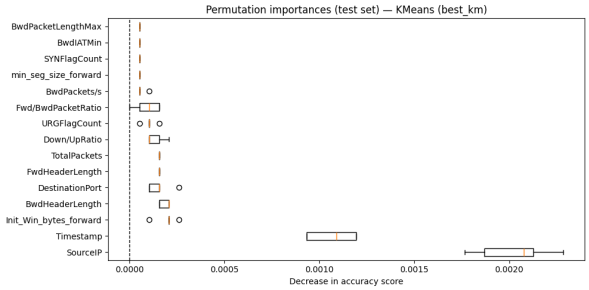
Figure **??** shows the Top-15 features for KMeans. The two most influential features are `SourceIP` and `Timestamp`. They produce the largest mean decrease in accuracy. Other relevant features include `Init_Win_bytes_forward`, `BwdHeaderLength`, `DestinationPort`, and `TotalPackets`. Several ratio-based and rate-based features also appear in the ranking (e.g., `Down/UpRatio`, `Fwd/BwdPacketRatio`, `BwdPackets/s`).

Figure **??** confirms this result. The importance distribution for `SourceIP` and `Timestamp` is clearly separated from the others, indicating a stable effect. Most remaining features have smaller values, but they are still consistently above zero.

To support interpretation, Figure **??** reports the standardized cluster means for the top features. It shows that the two clusters differ along the same dimensions highlighted by permutation importance. This is consistent with the coarse separation observed for KMeans in Section 4.1.



(a) Top 15 features (mean decrease in accuracy).



(b) Permutation importance distribution (test set).

(c) Standardized cluster means (top features).

Figure 23: Feature importance analysis for KMeans.

5.2.3 GMM (3 components)

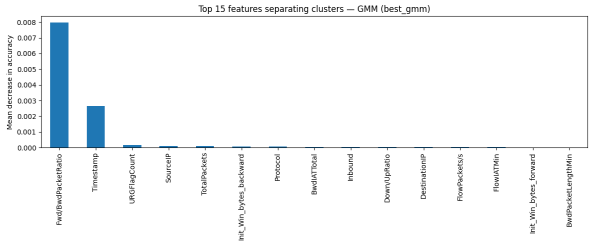
Figure **??** reports the Top-15 features for GMM. The most influential feature is `Fwd/BwdPacketRatio`. The second most important feature is `Timestamp`. Other features with non-negligible importance are `URGFlagCount`, `SourceIP`, `TotalPackets`, and `Protocol`. This suggests that GMM components are separated mainly by flow directionality patterns and temporal structure, with additional contributions from transport-level indicators.

Figure **??** shows the importance distributions on the test set. The dominance of `Fwd/BwdPacketRatio` is robust, since it has a much larger decrease in accuracy than the other features. The remaining features have smaller effects, but their median importance is still above zero.

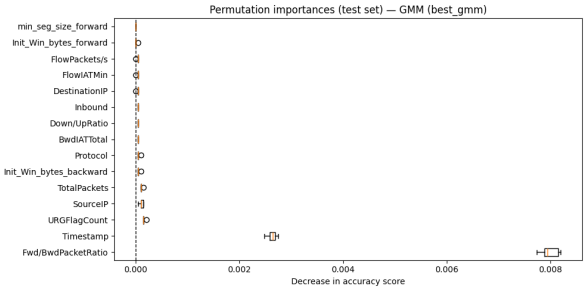
Figure **??** shows standardized cluster means for the top features. The clusters exhibit distinct profiles for `Fwd/BwdPacketRatio` and other high-ranked features. This provides a direct explanation of why GMM separates certain traffic groups more clearly than KMeans.

5.3 Attack Pattern Investigation

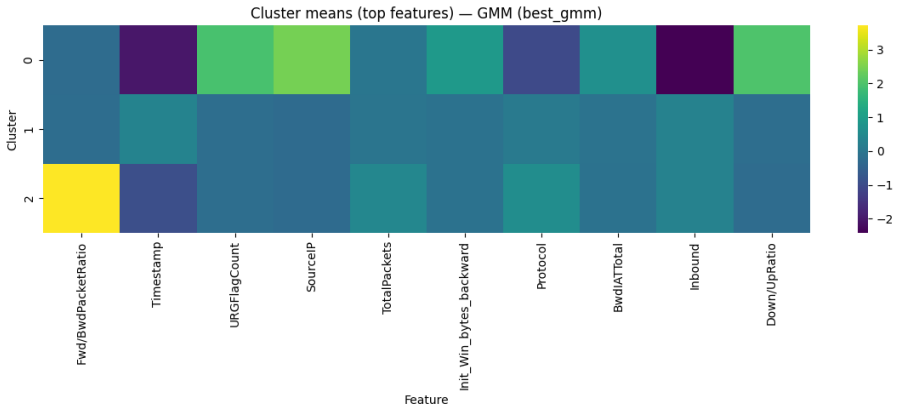
In this section we investigate whether each attack class contains multiple internal patterns (sub-attacks). We also analyze which attacks are more similar to each other. We rely on the per-class internal clustering results (best k by silhouette) and on the feature importance analysis from Section 4.2.



(a) Top 15 features (mean decrease in accuracy).



(b) Permutation importance distribution (test set).



(c) Standardized cluster means (top features).

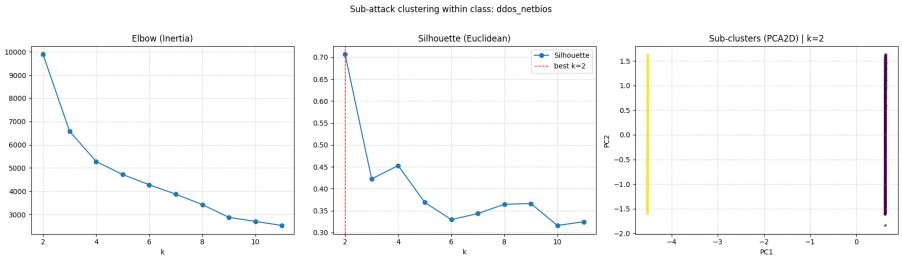
Figure 24: Feature importance analysis for GMM.

5.3.1 Sub-attacks inside the same GT class

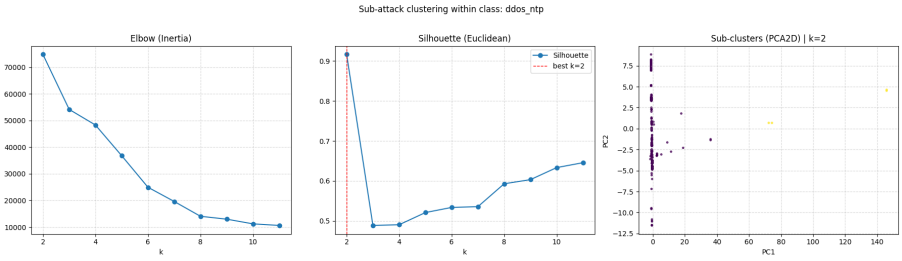
To detect sub-attacks, we cluster the samples of each GT class separately and select the best number of sub-clusters using the silhouette score. When the best k is greater than 1, the class is not homogeneous in the feature space. This indicates multiple internal patterns.

The results show that several attack classes have a clear sub-structure. For example, `ddos_netbios` (Figure **??**) has best $k = 2$ with a high silhouette score, and the PCA scatter plot shows two well-separated groups. This suggests two distinct patterns within the same class. A similar behavior is observed for `ddos_ntp` (Figure **??**), which achieves the highest silhouette score, and for `ddos_syn` (Figure **??**) and `ddos_tftp` (Figure **??**), which also prefer $k = 2$. Other classes show more complex internal variability. For instance, `ddos_snmp` (Figure **??**) and `ddos_mssql` select a larger k and obtain only moderate silhouette values. This indicates multiple sub-groups, but with weaker separation in the feature space.

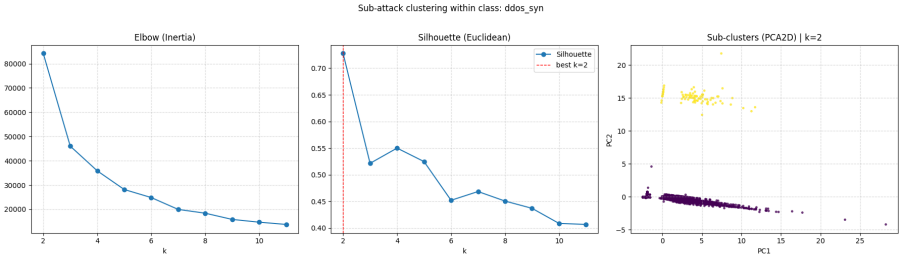
These sub-patterns are consistent with different traffic intensities and flow dynamics inside the same attack family. Based on Section 4.2, the main features that can drive such splits are rate and volume indicators (e.g., `FlowPackets/s`, `TotalPackets`), directionality ratios (e.g., `Fwd/BwdPacketRatio`, `Down/UpRatio`), and protocol or flag-related features (e.g., `Protocol`, `URGFlagCount`). We note that `Timestamp` can also separate patterns, but it may reflect dataset acquisition conditions rather than pure traffic behavior.



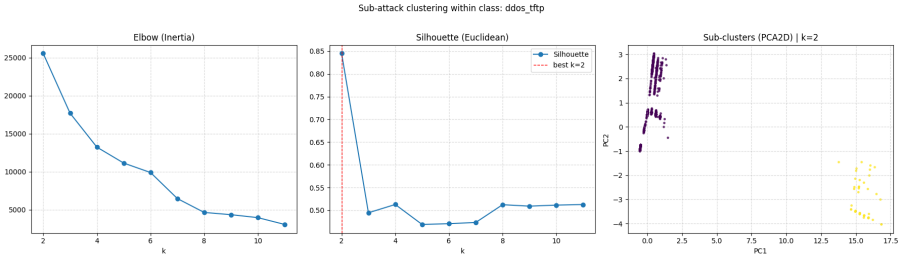
(a) Sub-clustering analysis for `ddos_netbios`.



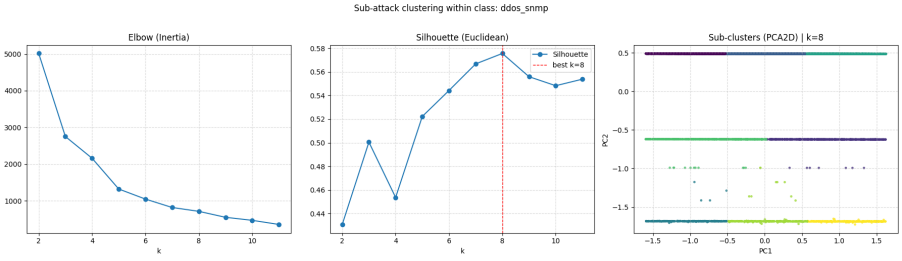
(b) Sub-clustering analysis for `ddos_ntp`.



(c) Sub-clustering analysis for `ddos_syn`.



(d) Sub-clustering analysis for `ddos_tftp`.



(e) Sub-clustering analysis for `ddos_snmp`.

Figure 25: Sub-attack clustering results for selected classes.

5.3.2 Which attacks are more similar and why

We compare attacks by measuring the absolute difference between class-wise feature medians. We report the top features with the largest median gaps. This highlights which features best discriminate two attack types.

UDP vs UDP-lag. Figure **??** compares `ddos_udp` and `ddos_udp_lag`. The largest differences appear in `FwdPacketLengthStd` and `Timestamp`, followed by traffic-rate and directionality features such as `FlowPackets/s` and `Fwd/BwdPacketRatio`. This suggests that the two attacks are similar at protocol level, but differ in timing and in packet-size variability. This is coherent with the idea of a "lag" variant, where the temporal pattern is a key element.

MSSQL vs SSDP. Figure **??** compares `ddos_mssql` and `ddos_ssdp`. The dominant separating feature is `FlowPackets/s`. `Timestamp` is also relevant. The remaining features have smaller gaps. This indicates that these two attacks are relatively similar in many statistics, and they mainly differ in traffic intensity. This matches their nature as high-volume UDP-based reflection/amplification traffic, with service-specific behavior affecting the effective rate.

LDAP vs MSSQL. Figure **??** compares `ddos_ldap` and `ddos_mssql`. `Timestamp` has the largest median gap, and `DestinationPort` is also discriminative. This is expected because the targeted service defines the destination port. Again, `Timestamp` may capture temporal acquisition effects, so service-related and traffic-statistics features provide a more stable interpretation.

Overall, the comparisons suggest that many reflection/amplification attacks are close to each other in the feature space. They share similar protocol and flow-level behavior. Differences mainly come from rate, directionality, and service-specific indicators such as ports.

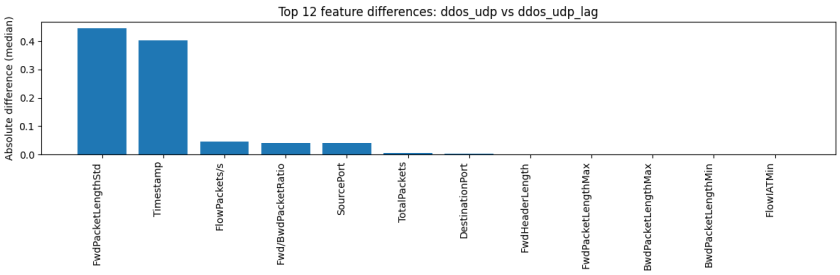
5.3.3 Benign vs malicious overlap

To evaluate benign–malicious similarity, we compare benign traffic against representative attacks using the same median-difference analysis.

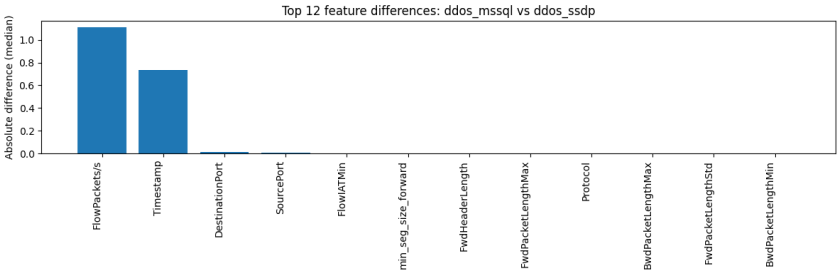
Figure **??** shows that `ddos_dns` differs from benign traffic mainly through directionality and protocol-related features. The strongest gaps include `Fwd/BwdPacketRatio`, `Inbound`, `Protocol`, and `Down/UpRatio`. These differences explain why DNS attacks often form a more distinct group in clustering.

Figures **??** and **??** show a similar trend for reflection attacks. `Inbound`, address/port features, and rate-related statistics are among the most discriminative. This indicates that benign traffic is generally well separated from these high-volume attacks in the considered feature space.

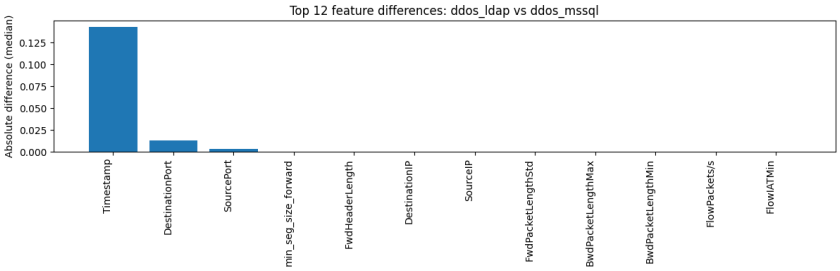
At the same time, earlier clustering results highlighted that some malicious flows can overlap with benign traffic, especially `ddos_ntp`. This suggests that not all attacks produce extreme feature values. Some sub-patterns may resemble benign behavior, which reduces separability.



(a) `ddos_udp` vs `ddos_udp_lag`.

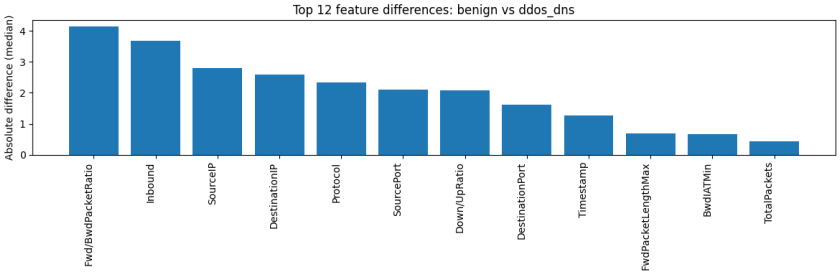


(b) `ddos_mssql` vs `ddos_ssdp`.

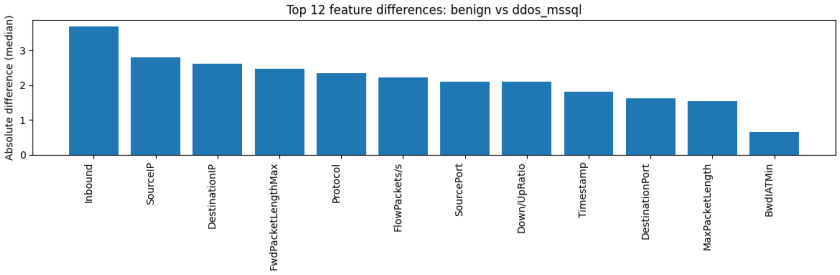


(c) `ddos_ldap` vs `ddos_mssql`.

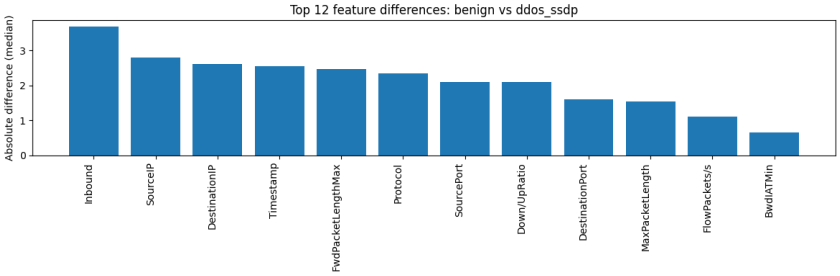
Figure 26: Top 12 feature differences between selected attack pairs (absolute median differences).



(a) `benign` vs `ddos_dns`.



(b) `benign` vs `ddos_mssql`.



(c) `benign` vs `ddos_ssdp`.

Figure 27: Top 12 feature differences between benign traffic and representative attacks (absolute median differences).